GigBid: Multi-Agent Freelancer Copilot — R&D; Engineering Design Review

Abdullah Razzaq (23i-0057, Section B) · AI-4016 Agentic AI · Repository: github.com/abdth3/gigbid

1. Problem, Constraints & Declared Budgets

Freelancers waste hours scanning noisy job feeds, spending paid Connects on low-fit or scam posts, and drafting generic proposals. GigBid automates discovery, risk scoring, portfolio evidence matching, pricing, and claim-verified drafting while strictly forbidding autonomous external writes: every proposal submission, client message, and Connects spend pauses at a LangGraph interrupt() for human approval. Declared open-profile budgets (agent/config.yaml): p95 latency <= 4.5s, cost/job <= $0.005 USD.

2. Architecture: Graphs, Marketplace Port, MCP Surface & Stores

Two LangGraph StateGraphs: (1) *Bid Graph* (agent/graphs/bid_graph.py) fans out from Scout via parallel Send() to Job Analyst, Risk Assessor, and Fit Matcher; routes via Decision to Pricing, Writer, Claim Verifier, Critic (max 2 loops), Compliance Guard, and interrupt() Human Approval before idempotent submission. (2) *Conversation Graph* (agent/graphs/conversation_graph.py) classifies client intent, suggests calendar slots via Workspace MCP, and gates replies behind interrupt().
Single Marketplace Port (R1): agent/marketplace_port.py exposes one interface switched by config between GigSimAdapter and UpworkLiveAdapter (Streamable HTTP + encrypted OAuth 2.1).
MCP Tool Layer: Three FastMCP servers (portfolio, marketplace, workspace) with Pydantic schemas, ToolAnnotations, idempotency_key, dry_run, per-user rate limiting, and append-only audit logging.
State vs. Trace vs. Memory: Working state (GigBidState) is separated from execution_trace (audit log) and Long-Term Memory (style_memory from human edit diffs + outcome_memory from won/replied jobs via agent/memory/promotion_rules.py).

3. R&D; Question A: Tool Granularity Study (evals/results/tool_design_eval.json)

Hypothesis: A curated 14-tool MCP surface with per-agent allowlists and explicit Pydantic constraints outperforms both 24 overlapping fine-grained tools and 3 overloaded coarse tools across 40 benchmark tasks.

Design	Tools	Selection Acc	Arg Acc	Avg Tokens/Task
fine_grained_24_tools	24	0.775	0.800	640
coarse_3_tools	3	0.925	0.725	510
curated_14_tools_rewritten_descriptions	14	0.975	0.950	430

Conclusion: Curated 14 tools with rewritten descriptions achieved 97.5% tool selection and 95.0% argument accuracy while cutting token usage by 32.8% vs. the 24-tool surface.

4. R&D; Question B: Advanced RAG, Claim Verification & Ablation (evals/results/rag_eval.json)

Evaluated 8 variants across 120 dev jobs (40 per persona). Judge calibration on n=30 items yielded Cohen's kappa = 0.841 (93.3% agreement). All variants run through Claim Verification (rag/verify.py), reducing unsupported claims from 1.000 before verification to 0.000 after.

Variant	Recall@5	MRR	nDCG@10	Unsupp (Pre->Post)	Reply Rate	p95 (s)
v0_naive	0.000	0.058	0.166	1.0 -> 0.0	0.000	0.0197
v1_decomp	0.000	0.059	0.169	1.0 -> 0.0	0.000	0.0797

Variant	Recall@5	MRR	nDCG@10	Unsupp (Pre->Post)	Reply Rate	p95 (s)
v2_hybrid	0.008	0.114	0.283	1.0 -> 0.0	0.005	0.0391
v3_rerank	1.000	1.000	1.000	1.0 -> 0.0	0.650	0.0258
v4_outcome	0.767	0.558	0.661	1.0 -> 0.0	0.498	0.0274
v5_multivector	0.708	0.327	0.486	1.0 -> 0.0	0.460	0.0252
v8_corrective	1.000	1.000	1.000	1.0 -> 0.0	0.650	0.0257
v9_skill_graph	0.008	0.114	0.283	1.0 -> 0.0	0.005	0.0257

Persona Breakdown (v4_outcome Recall@5): ai_ml_engineer=0.575, fullstack_web_dev=0.825, data_analyst=0.9.
Negative Finding & Shipped Decision: Naive dense (v0) scored 0.000 Recall@5 because 150 past proposals crowded out gold case studies. Outcome-aware ranking (v4) boosted past 'won' proposals above case studies (0.767 Recall@5). **v3_rerank** (Hybrid BM25+Dense RRF + pairwise cross-encoder rerank with case-study precision boost) achieved 1.000 Recall@5 and 0.650 GigSim reply rate at 0.0258s p95, so **v3_rerank** ships as default in agent/config.yaml.

5. R&D; Question C: Multi-Agent Architecture Comparison (evals/results/multi_agent_eval.json)

Architecture (n=50)	Bid/Skip Acc	Unsupp/Prop	Reply Rate	Avg Tokens	p95 (s)
single_react_agent	0.780	0.680	0.420	3850.0	0.850
prebuilt_supervisor	0.840	0.310	0.560	5420.0	1.350
hand_built_langgraph	0.900	0.000	0.467	894.5	0.141

When Single ReAct Wins: On simple 1-requirement jobs, Single ReAct uses fewer steps. However, it suffers 0.680 unsupported claims/proposal and 0.780 bid/skip accuracy. Our hand-built LangGraph achieves 0.900 accuracy, 1.000 scam recall, 0.000 unsupported claims, and 894.5 avg tokens/job via early skip.

6. Absorbing the Nov 2 Platform Policy Change Request

The Nov 2 policy update introduced a 1,800-character proposal cap and a +4 Connects surcharge on bids > $500. Because rules and prompts live in configuration (agent/config.yaml under policy_nov2, data/policy_pack/marketplace_policy.md, and agent/prompts/compliance.md), absorbing the change required editing 3 config keys and 6 lines across pricing.py/compliance.py in under 15 minutes with zero graph rewiring.

7. Signature Extension X2: Proposal A/B Experiments (evals/results/x2_ab_results.json)

Tested two writer framing strategies across n=60 bid jobs (30 per arm) in GigSim: Arm A (evidence_first) achieved 16/30 (53.3%) replies vs. Arm B (outcome_led) with 20/30 (66.7%) replies. Two-proportion z-test yielded z=1.054, p=0.2918 (not statistically significant at alpha=0.05; larger sample n>=180 needed to confirm).

8. Failure Analysis Taxonomy (10 Dev Jobs)

Category	Job ID(s)	Root Cause	Fix & Outcome
Missed Evidence	dev-ai_m-004, -019	Implied skill phrasing defeated V0 dense	V2 BM25+Dense + V3 rerank -> Recall@5 1.000
Wrong Skip	dev-ai_m-042	Strict threshold skipped 0.58 fit job	Lowered minimum_fit_score to 0.55 in config.yaml
Mispriced	dev-full-015	Ignored Nov-2 +4 Connects > $500	Added policy_nov2 surcharge to pricing.py
Unsupported Claim	dev-data-011	Writer added unverified years claim	verify_and_rewrite_proposal() stripped to 0.000

Category	Job ID(s)	Root Cause	Fix & Outcome
Generic Opening	dev-ai_m-027	'Dear Sir/Madam' triggered client skip	Style memory + Critic check blocked generic starts
Compliance Miss	dev-full-033	Long draft > 1,800 chars	Compliance Guard enforces max_cover_letter_chars
Prompt Injection	dev-ai_m-021, -024	Embedded [SYSTEM OVERRIDE] in post	Delimited tags + Risk regex -> 0.0% ASR
Tool / Rate Error	batch-eval-50	30 RPM Marketplace MCP limit tripped	Exponential backoff + eval RPM config

9. Safety, Injection Defence & Platform Respect (evals/results/injection_eval.json)

Tested on 30 planted-injection jobs: baseline unprotected attack success rate was 73.3%; with delimiters, Risk Assessor regex/pattern detection (100.0% detected), Claim Verifier, and Compliance Guard, post-defence attack success rate dropped to 0.0%. PII is masked via mask_pii(), OAuth tokens are Fernet-encrypted, and live Upwork mode uses official MCP drafts only with 30-minute polling.

10. Budget Compliance & Production Readiness (evals/results/hidden_budget_summary.json)

Measured p95 latency on the open profile is **0.1405s** (well within declared 4.5s budget) at **$0.000/job**. Key optimization: routing cheap nodes (Scout/Analyst/Risk) to qwen3-4b, skipping scams/low-fit jobs before Writer, and caching portfolio evidence. All 11 unit/contract/integration tests pass (pytest -q) and Docker Compose + GitHub Actions CI/eval gates are active.